Elman, Jordan and Multi-Recurrent Neural Networks for Univariate Time Series Forecasting under Regime Changes

M. Bennett, 26913348
26913348@sun.ac.za

***Abstract*—This report evaluates three simple recurrent neural networks: the Elman, Jordan, and multi-recurrent networks, for one-step-ahead forecasting on five univariate time series from the M4 competition. Two of the series appear stationary, while the remaining three contain a unit root and abrupt regime changes. Performance was assessed using five-fold expanding-window cross-validation. For each fold, Optuna used a tree-structured Parzen estimator with median pruning to tune the hidden layer size, input window length, learning rate, weight decay, and batch size. Early stopping was applied using validation loss, and the best configuration from each fold was retrained with 10 random seeds before evaluation on the test block.**

On the hourly series, the Elman network achieved a root mean squared error (RMSE) of 0.120 and the multi-recurrent network 0.123; their difference was not significant, but both significantly outperformed the Jordan network, which obtained an RMSE of 0.150. All three networks reduced the naive last-value forecast error by more than 80% on the hourly series and by 3% to 6% on the weekly series. On the three random-walk-like series, however, none of the networks differed from the naive forecast by more than 1.3% in mean relative error. Across all five datasets, the Elman network had the lowest mean rank at 1.4, although the Friedman test found no significant overall difference between the three architectures ($p = 0.165$).

All three networks recovered from abrupt level shifts at roughly the same rate as the naive forecast. In contrast, none adapted to a change in the underlying dynamics without retraining. When retraining was triggered by an increase in rolling error, the error of the Elman and multi-recurrent networks was reduced by about half, whereas the Jordan network improved by only 2% to 3%. The experiments also showed that scaling raw series containing large level shifts reduced the accuracy of all three networks by 25% to 46% relative to the naive forecast.

I. Introduction

Forecasting the next value in a time series is a common task across many industries. Recurrent neural networks are well suited to this problem because their recurrent connections allow information from earlier inputs to influence later predictions [7], [8]. Simple recurrent neural networks use a single hidden layer together with a context layer that stores information from the previous time step. The three networks considered in this report differ in what this context layer stores. The Elman network stores the previous hidden state [7], the Jordan network stores the previous output [14], and the multi-recurrent network stores both at several rates of decay [22].

Real-world time series are often non-stationary, with their level, trend or dynamics changing over time. When such a change occurs abruptly, it can be described as a regime change, while the broader phenomenon is referred to as concept drift [10]. A model trained before such a change has not seen the new behaviour, so its response to regime changes is an important part of its evaluation.

This report evaluates the Elman, Jordan and multi-recurrent networks on five univariate time series. The series are first analysed for stationarity and pre-processed without using future information. The networks are then tuned and trained under the same conditions using time-series cross-validation, and their performance is compared using statistical tests. Their response to regime changes is also measured. An additional experiment examines whether retraining after a rise in forecasting error helps the networks adapt to a new regime.

The networks outperformed the naive last-value forecast only on the two series with repeating patterns, H279 and W8. On the hourly series, the Elman and multi-recurrent networks were significantly more accurate than the Jordan network. Across all five datasets, however, no network was significantly better than the other two. After a change in dynamics, adaptation required retraining, and the Elman and multi-recurrent networks benefited substantially more than the Jordan network.

The remainder of the report is organised as follows. Section II introduces the forecasting problem, the three networks, time-series cross-validation, hyperparameter optimisation and the statistical tests. Section III describes the datasets and pre-processing. Section IV defines the empirical procedure. Section V presents and discusses the results. Section VI concludes the report.

II. Background

A. Time Series Forecasting and Stationarity

A univariate time series consists of a sequence of observations $(y_1, y_2, \ldots, y_n)$, where y_t denotes the value observed at time step t. In one-step-ahead forecasting, the aim is to predict y_t using only observations available up to time $t-1$. A simple baseline is the naive forecast,

$$\hat{y}_t = y_{t-1}, \tag{1}$$

which uses the most recent observation as the prediction for the next one. For a random walk, this is the optimal one-step forecast because the expected next value is equal to the

current value [13]. It therefore provides a useful benchmark for evaluating more complex forecasting models.

A time series is stationary when properties such as its mean and variance remain stable over time. Two commonly used tests for stationarity are the augmented Dickey-Fuller (ADF) test and the Kwiatkowski-Phillips-Schmidt-Shin (KPSS) test. The ADF test has the null hypothesis that the series contains a unit root, while the KPSS test uses stationarity as its null hypothesis [6], [16]. A small ADF p-value therefore provides evidence against a unit root, whereas a small KPSS p-value provides evidence against stationarity. A series with a unit root can often be made stationary by first differencing, replacing y_t with $y_t - y_{t-1}$ [13]. A logarithmic transform can also be useful when the magnitude of the fluctuations increases with the level of the series, because it can help stabilise the variance [13].

Gama *et al.* defined concept drift as a change over time in the joint distribution of the inputs and target of a prediction problem [10]. In time-series forecasting, the previous observations form the inputs and the next observation is the target. An abrupt form of concept drift is referred to here as a regime change, with the time at which the change occurs called the break point. Gama *et al.* also differentiated between blind adaptation, where a model is updated at fixed intervals, and informed adaptation, where an update is triggered only after a change detector signals that the data distribution may have shifted [10]. A simple detector can compare recent forecasting error with the error level normally produced by the model.

B. Simple Recurrent Neural Networks

A simple recurrent neural network extends a feedforward neural network by adding a context layer [8]. This context stores information from the previous time step and feeds it back into the hidden layer together with the next input. The connections used to copy information into the context layer are fixed, while the weights from the input, context and hidden layers are learned during training.

For forecasting, the network processes a window containing the ℓ most recent observations, one observation at a time. Let x_τ denote the input at step τ, where $\tau = 1, \ldots, \ell$, and let q denote the number of hidden units. The hidden layer uses the hyperbolic tangent activation function, while the output layer is linear so that the prediction is not restricted to a fixed range. The output produced at the final step, o_ℓ, is used as the forecast of the next value. The Elman, Jordan and multi-recurrent networks differ mainly in the information that is stored and fed back through their context mechanisms, as illustrated in Fig. 1.

1) Elman Network: The Elman network uses a context layer that stores a copy of the hidden state from the previous step [7]. Its hidden state is calculated as

$$\mathbf{h}_\tau = \tanh\left(\mathbf{w}x_\tau + \mathbf{U}\mathbf{c}_\tau + \mathbf{b}\right), \qquad \mathbf{c}_\tau = \mathbf{h}_{\tau-1}, \tag{2}$$

where $\mathbf{w}$ is the vector of input weights, $\mathbf{U}$ is the $q \times q$ context-weight matrix, $\mathbf{b}$ is the hidden bias vector and $\mathbf{c}_\tau$ is the context vector. The context is initialised to zero. The network output is

$$o_\tau = \mathbf{v}^\top \mathbf{h}_\tau + d, \tag{3}$$

where $\mathbf{v}$ is the output-weight vector and d is the output bias.

The Elman context contains q units, one for each hidden unit. The learned matrix $\mathbf{U}$ determines how strongly information from the previous hidden state affects the current one. For example, a large diagonal element allows a hidden unit to retain more of its previous activation. In this sense, the Elman network learns how information should be carried forward through time. The network contains $q^2 + 3q + 1$ trainable parameters.

2) Jordan Network: The Jordan network differs from the Elman network by feeding the previous output back into the context layer [14]. With a single output, the context consists of one unit, e_τ. The hidden state is

$$\mathbf{h}_\tau = \tanh\left(\mathbf{w}x_\tau + \mathbf{a}e_\tau + \mathbf{b}\right), \qquad e_\tau = \mu e_{\tau-1} + o_{\tau-1}, \tag{4}$$

where $\mathbf{a}$ is the vector of context weights and μ is the fixed self-connection weight. The context starts at zero, and the output is calculated using the same linear output equation as above. Expanding the recurrence gives

$$e_\tau = o_{\tau-1} + \mu o_{\tau-2} + \mu^2 o_{\tau-3} + \ldots, \tag{5}$$

so the context represents an exponentially decaying combination of earlier outputs. When $\mu = 0$, only the most recent output is retained. A non-zero self-connection therefore extends the effective memory without introducing additional trainable weights. However, because the context contains only a single value, the amount of information that can be retained at each step is limited. The Jordan network has $4q+1$ trainable parameters.

3) Multi-Recurrent Network: The multi-recurrent network combines hidden-state and output feedback [22]. It also uses several memory banks, each with a different fixed decay ratio. Let p be the number of banks and let $\rho_i \in [0, 1)$ denote the ratio for bank i. The hidden-state and output memories are updated as

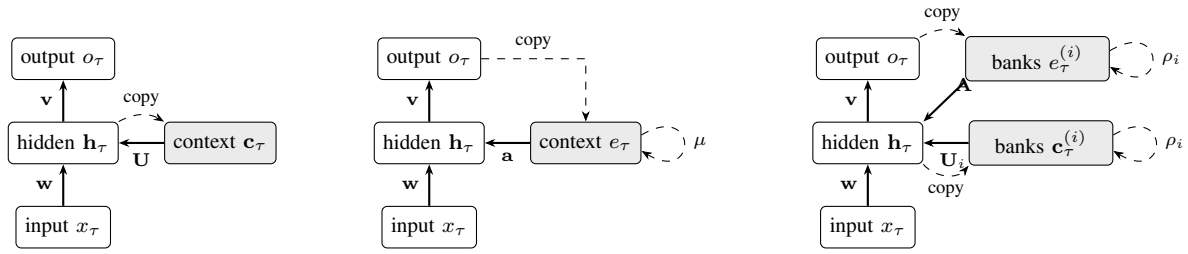
$$\mathbf{c}_\tau^{(i)} = \rho_i\, \mathbf{c}_{\tau-1}^{(i)} + (1 - \rho_i)\, \mathbf{h}_{\tau-1}, \tag{6}$$

$$e_\tau^{(i)} = \rho_i\, e_{\tau-1}^{(i)} + (1 - \rho_i)\, o_{\tau-1}. \tag{7}$$

A bank with $\rho_i = 0$ stores only the most recent feedback, while a value closer to one causes the bank to change more slowly and preserve information from further back in time. All memory banks contribute to the next hidden state,

$$\mathbf{h}_\tau = \tanh\Big(\mathbf{w}x_\tau + \sum_{i=1}^{p} \mathbf{U}_i \mathbf{c}_\tau^{(i)} + \mathbf{A}\mathbf{e}_\tau + \mathbf{b}\Big), \tag{8}$$

where $\mathbf{U}_i$ contains the weights from hidden-memory bank i, $\mathbf{e}_\tau = (e_\tau^{(1)}, \ldots, e_\tau^{(p)})$ contains the output-memory banks and $\mathbf{A}$ contains their weights into the hidden layer. By using several decay rates, the network can access information from both recent and more distant time steps. The trade-off is a larger number of trainable parameters, equal to $pq^2 + (p+3)q + 1$.



(a) Elman network (b) Jordan network (c) Multi-recurrent network

Fig. 1. Architectures of the three simple recurrent neural networks. Solid arrows are trained weights. Dashed arrows are fixed connections.

4) Training: The weights of all three networks are trained using backpropagation through time [23]. In this approach, the recurrent network is unrolled across the ℓ steps of the input window, producing a feedforward structure in which the same weights are reused at every step. The loss gradient can then be calculated by applying ordinary backpropagation through this unrolled structure. When the network is unrolled over many steps, the gradients may become extremely large or shrink towards zero, producing exploding or vanishing gradients. Pascanu *et al.* showed that rescaling a gradient when its norm exceeds a chosen threshold can be used to control exploding gradients [19].

Adam is a gradient-based optimisation method that adjusts the update size of each weight independently [15]. It maintains exponentially decaying averages of both the gradient and the squared gradient and uses these quantities to scale the update applied to each parameter. Weights that repeatedly receive large gradients are therefore updated more cautiously, while weights associated with smaller gradients can take relatively larger steps. The learning rate η controls the overall magnitude of these updates.

Weight decay introduces the penalty $\frac{\lambda}{2}\|\boldsymbol{\theta}\|^2$ into the loss function, where $\boldsymbol{\theta}$ represents the vector of trainable weights and λ is the weight-decay coefficient. This penalty discourages unnecessarily large weights by pulling weakly supported parameters towards zero, which reduces the effective complexity of the network. Early stopping provides a second mechanism for limiting overfitting. Training is stopped when the validation loss has failed to improve for a specified number of epochs, after which the weights corresponding to the lowest observed validation loss are restored [20].

C. Cross-Validation for Time Series

Standard k-fold cross-validation typically assigns observations randomly to different folds. For time-series data, this can cause future observations to appear in the training set while earlier observations appear in the test set. Bergmeir and Benítez showed that the dependence between neighbouring time-series observations makes this form of random splitting problematic and recommended evaluation procedures that preserve temporal order [2]. Tashman reviewed methods for out-of-sample forecast evaluation and recommended rolling-origin evaluation [21]. In this approach, the forecast origin advances through the series. For each fold, all observations before the origin are used for training and the observations immediately following the origin are used for testing. As the origin moves forward, the training block grows, which is why this method is also referred to as expanding-window cross-validation. This ensures that every test observation occurs after all observations used for training, matching the order that would occur in real forecasting.

D. Hyperparameter Optimisation

Hyperparameters, including quantities such as the number of hidden units and the learning rate, must be selected before the final model is trained. Hyperparameter optimisation searches for a combination of these values that minimises the validation loss. Grid search tests every possible combination drawn from a predefined set of values for each hyperparameter. The number of combinations therefore increases rapidly as more hyperparameters are added. Bergstra and Bengio showed that random search can find configurations that are as good as or better than those found by grid search using fewer evaluations, since only a subset of hyperparameters often has a strong effect on performance [4]. However, both grid search and random search choose new configurations without using information from the results of earlier evaluations.

1) Tree-Structured Parzen Estimator: The tree-structured Parzen estimator (TPE), introduced by Bergstra *et al.*, is a sequential model-based optimisation method that uses the outcomes of previous evaluations when selecting new hyperparameter configurations [3]. Let $\boldsymbol{\phi}$ represent a hyperparameter configuration and let $f(\boldsymbol{\phi})$ denote the validation loss obtained from that configuration. Once several configurations have been evaluated, TPE divides them into a good group and a poor group using a threshold f^*. The threshold is chosen so that a fraction γ of the observed configurations satisfies $f(\boldsymbol{\phi}) < f^*$. TPE then estimates two probability densities over the hyperparameter space,

$$l(\boldsymbol{\phi}) = p\left(\boldsymbol{\phi} \mid f(\boldsymbol{\phi}) < f^*\right), \qquad g(\boldsymbol{\phi}) = p\left(\boldsymbol{\phi} \mid f(\boldsymbol{\phi}) \geq f^*\right). \tag{9}$$

Both $l(\boldsymbol{\phi})$ and $g(\boldsymbol{\phi})$ are Parzen estimators. A Parzen estimator places a probability kernel, such as a truncated Gaussian, around each previously observed hyperparameter value and combines these kernels to form a density estimate. Bergstra *et al.* showed that expected improvement over the threshold

f^* can be expressed in terms of these two densities, which leads to selecting configurations with a large value of the ratio $l(\boldsymbol{\phi})/g(\boldsymbol{\phi})$ [3]. TPE therefore samples candidate configurations from $l(\boldsymbol{\phi})$ and selects candidates that have a high ratio between the good and poor densities. Such a configuration is more similar to previously successful trials than to unsuccessful ones. After each evaluation, the density estimates are updated, allowing the search to concentrate increasingly on promising areas while still exploring alternatives. The first TPE trials are selected randomly because too few observations are initially available to construct useful density estimates.

2) Median Pruning: Pruning terminates trials that appear unlikely to become competitive before they complete all training epochs. During training, a trial reports intermediate measurements, such as the validation loss obtained after each epoch. The median pruner stops a trial at epoch s when the best value achieved by that trial so far is worse than the median intermediate value reported at the same epoch by previously completed trials [1]. This reduces the amount of computation spent on hyperparameter configurations that are unlikely to outperform the configurations already evaluated. Because a pruned trial does not produce a final validation loss, TPE treats completed trials as more informative when separating observations into good and poor groups.

Optuna is an open-source framework for hyperparameter optimisation that provides implementations of both TPE sampling and median pruning [1]. The hyperparameter search space is defined inside an objective function. A study repeatedly evaluates this objective, using a sampler to propose each new configuration and a pruner to terminate trials whose intermediate performance is not promising.

E. Statistical Comparison of Several Models

Demšar recommended the use of non-parametric statistical tests when comparing learning algorithms because these tests do not require the performance values to follow a normal distribution [5]. The Friedman test is used to compare several models across matched blocks [9]. Within each block, the competing models are ranked from best to worst, with the best-performing model receiving rank one. Let ν denote the number of blocks and let $\bar{r}_j$ denote the mean rank of model j across those blocks. For three models, the Friedman statistic is

$$\chi_F^2 = \frac{12\nu}{3 \cdot 4} \left(\sum_{j=1}^{3} \bar{r}_j^2 - \frac{3 \cdot 4^2}{4} \right), \tag{10}$$

which follows a χ^2 distribution with two degrees of freedom under the null hypothesis that all models perform equally. When the Friedman test rejects the null hypothesis, a post-hoc test identifies which pairs of models differ. The Wilcoxon signed-rank test compares two models over matched blocks, using the ranks of the absolute differences between the two models [24]. Three models give three pairwise tests, which increases the chance of a false rejection. The Holm procedure controls the chance of any false rejection over a family of tests [11]. The procedure sorts the p-values from smallest to largest and multiplies the smallest by three, the second smallest by two and the largest by one. Each adjusted p-value is then raised, where necessary, to at least the previous adjusted p-value.

TABLE I
DATASETS AND STATIONARITY TESTS

Series	Frequency	Length	ADF p	KPSS p	Break points
H279	hourly	1,008	< 0.001	0.01	none
W8	weekly	1,636	< 0.001	0.10	none
D686	daily	4,211	0.43	0.01	3,626
M6864	monthly	2,677	≈ 1.0	0.01	2,430
M26487	monthly	909	0.50	0.01	303, 606

III. DATASETS AND PRE-PROCESSING

This section describes the five datasets and the pre-processing of each dataset. Section III-A describes the datasets and the stationarity of each series. Section III-B describes and justifies the pre-processing steps.

A. Datasets

The five datasets are single series from the M4 forecasting competition, which contains 100,000 series from six domains and six sampling frequencies [18]. The series were selected to cover four sampling frequencies, both stationary and non-stationary behaviour, and three different kinds of regime change. For each series, the M4 training and test parts were joined into one series. Table I summarises the datasets, and Fig. 2 shows the five series.

H279 is an hourly series with a smooth and regular 24-hour cycle between about 10 and 22. The series has no trend and no break point. The ADF test rejected a unit root, while the KPSS test rejected level stationarity at the 0.01 level. The amplitude of the daily cycle grows slowly over the first half of the series and shrinks over the second half, which explains the result of the KPSS test. H279 is therefore stationary in the mean but not in the variance, and is the easiest of the five datasets.

W8 is a weekly series between about 1,500 and 14,000, with a yearly peak roughly every 52 weeks and no long-term trend. Sharp single-week spikes occur throughout the series. The ADF test rejected a unit root and the KPSS test did not reject stationarity, so W8 is stationary. The high noise level makes W8 harder to forecast than H279.

D686 is a daily series that behaves like a random walk with an upward drift, from about 4,000 to about 19,000. At $t = 3{,}626$, the series falls to about 4,500 in a single step. The fall is about 60 times the standard deviation of the daily changes. The ADF test did not reject a unit root and the KPSS test rejected stationarity, so D686 is non-stationary.

M6864 is a monthly series that decreases in piecewise-linear segments between about 500 and 2,200. At $t \approx 2{,}430$, the level roughly triples and large oscillations appear that the series did not have before. M6864 is therefore the clearest example of concept drift among the datasets, because the dynamics change in addition to the level. Both tests indicated non-stationarity.

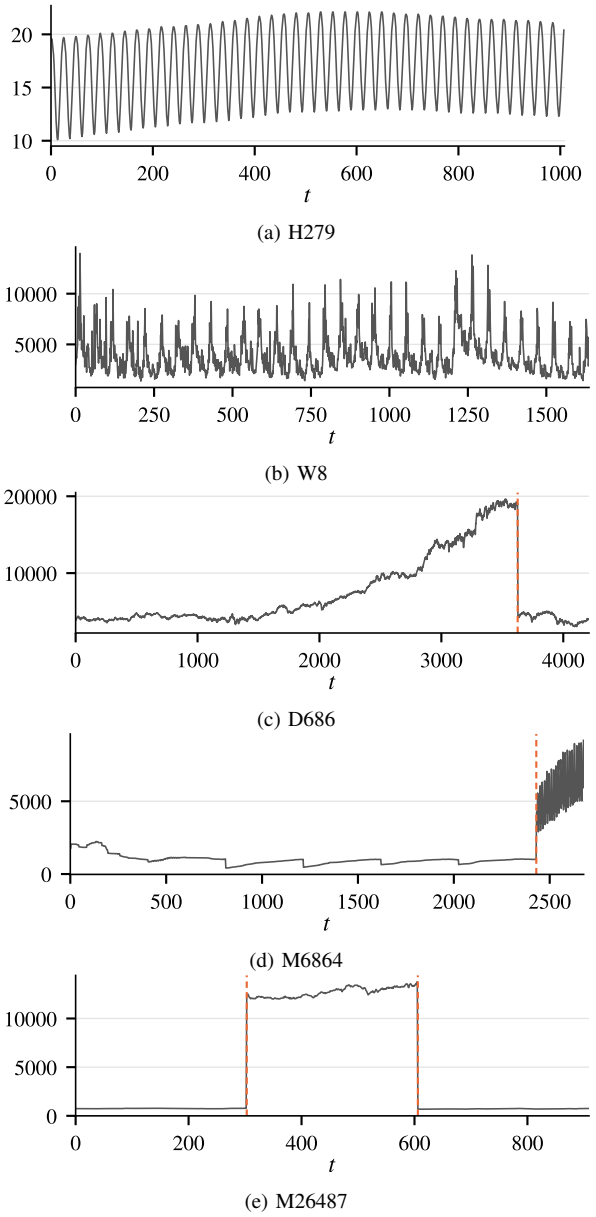


Fig. 2. The five series. Dashed lines mark the break points.

The length of 2,677 points is unusual for a monthly series, so the frequency label of M4 does not necessarily reflect the true sampling rate.

M26487 is a monthly series with three regimes. The series stays at about 700 until $t = 303$, jumps to about 12,000, drifts up to about 13,500 and falls back to about 700 at $t = 606$. Each jump is about 20 times the standard deviation of the monthly changes. The last regime repeats the first regime. Both tests indicated non-stationarity.

B. Pre-Processing

The pre-processing of every series follows the same pipeline of five steps. The first step is an optional logarithmic transform. The second step is optional first differencing. The third step scales the result to a fixed range. The fourth step cuts the scaled series into input windows of length ℓ, each paired with the next value as the target. The fifth step, applied after forecasting, inverts the scaling, differencing and logarithm, so that all errors are measured in the original units of the series. Only the choice of transform and the default window length differ between the datasets. Table II lists the settings.

TABLE II
PRE-PROCESSING SETTINGS PER DATASET

Series	Logarithm	Differencing	Scaling	Default ℓ
H279	no	no	min-max	24
W8	yes	no	min-max	52
D686	yes	yes	min-max	14
M6864	yes	yes	min-max	12
M26487	yes	yes	robust, clipped	12

1) Transforms: The three series with a unit root, namely D686, M6864 and M26487, were differenced, because first differencing removes a unit root [13]. The logarithm was applied before differencing, so that the differences are relative changes rather than absolute changes. The daily changes of D686, for example, are larger when the level is 19,000 than when the level is 4,000. Relative changes are comparable across levels. H279 and W8 were not differenced, because the ADF test rejected a unit root for both series. W8 was log-transformed, because the spikes of W8 span an order of magnitude and the logarithm compresses large values more than small values. H279 was left untransformed, because the values of H279 stay between 10 and 22.

Seasonal differencing was not applied. The seasonal pattern was instead left for the networks to learn, by setting the default window length to one seasonal period where the period is known. The default window lengths are 24 hours for H279, 52 weeks for W8, 14 days for D686 and 12 months for M6864 and M26487. The window length was also a tuned hyperparameter.

2) Scaling: The inputs were scaled to the range $[-1, 1]$, which matches the output range of the hyperbolic tangent. LeCun *et al.* showed that inputs on a small, common scale speed up gradient-based training [17]. Let $\tilde{y}_t$ denote the transformed value. Min-max scaling computes

$$z_t = 2\,\frac{\tilde{y}_t - \tilde{y}_{\mathrm{lo}}}{\tilde{y}_{\mathrm{hi}} - \tilde{y}_{\mathrm{lo}}} - 1, \tag{11}$$

where $\tilde{y}_{\mathrm{lo}}$ and $\tilde{y}_{\mathrm{hi}}$ are the minimum and maximum of the transformed training values. The scaling constants were computed separately for every fold, from the training block of that fold only. Constants computed from the whole series would include information from the validation and test blocks, which would leak future information into training.

M26487 required a different treatment, because its two jumps become single extreme values after differencing. The logarithmic change at $t = 303$ is $+2.77$, while the median absolute monthly change is about 0.004. Every training block contains the jump at $t = 303$. With min-max scaling, the jump defines the range of (11), and 90% of the changes fall into less

Algorithm 1 Empirical procedure for one dataset and one network

for each of the five expanding-window folds **do**
  Create a TPE study with a median pruner
  for each of the 30 trials **do**
    Let TPE suggest a configuration of the five hyperparameters
    Pre-process the series with the suggested window length, with scaling constants from the training block only
    Train the network on the training block with early stopping on the validation block, and report the validation loss after every epoch to the pruner
    Return the lowest validation loss to the study, unless the pruner stopped the trial
  end for
  Select the configuration with the lowest validation loss
  for each of 10 random seeds **do**
    Initialise a new network and train the network with the selected configuration, with early stopping on the validation block
    Forecast every value of the test block, and record the errors in the original units
    Forecast every value after the training block, for the regime change measures
  end for
end for

than 1% of the range $[-1, 1]$. Robust scaling was therefore used for M26487. Robust scaling sets $\tilde{y}_{\mathrm{lo}}$ and $\tilde{y}_{\mathrm{hi}}$ in (11) to the 1st and 99th percentiles of the training values instead of the minimum and maximum. The jump then maps to a scaled value between about $+200$ and $+250$, depending on the fold. A single target of that size would dominate the squared-error loss. Scaled values were therefore clipped to $[-3, 3]$. Clipping only affects the values that the network sees. All errors were still measured against the true, unclipped values. The other four datasets have no extreme single changes in any training block, so those datasets kept min-max scaling.

IV. Empirical Procedure

Algorithm 1 summarises the experimental procedure for one dataset and one network. The procedure was implemented in Python with PyTorch[1] and Optuna 4.9[2]. The code is available in a Git repository[3].

A. Network and Training Settings

All three networks used the architectures of Section II-B, with one input unit, one hidden layer of q hyperbolic tangent units and one linear output unit. The Jordan network used a self-connection weight of $\mu = 0.5$, following the self-connections of the original design [14]. The multi-recurrent network used $p = 4$ memory banks with ratios $\rho = (0, 0.25, 0.5, 0.75)$, following the bank structure of Ulbricht [22]. Every input window was processed as an independent sequence, with the context layer set to zero at the first step. The memory of every network was therefore limited to the ℓ steps of the input window. The weights were initialised from the uniform distribution on $[-1/\sqrt{n_{\mathrm{in}}}, 1/\sqrt{n_{\mathrm{in}}}]$, where n_{in} is the number of inputs to the layer, which is the default of PyTorch.

The loss was the mean squared error (MSE) between the output at the last step of each window and the scaled target, averaged over a mini-batch. The outputs at the earlier steps of the window were not part of the loss. The earlier outputs of the Jordan network and the multi-recurrent network were therefore free internal signals and not forecasts. The gradient was computed with backpropagation through time over the full window. The gradient was rescaled whenever the norm of the gradient exceeded 1.0 [19]. The weights were updated with Adam, with the default decay rates of 0.9 and 0.999 for the two averages [15]. Weight decay was applied as the penalty of Section II-B.

Training ran for at most 300 epochs. After every epoch, the MSE on the validation block was computed. Training stopped when the validation MSE had not improved for 20 consecutive epochs, and the weights of the epoch with the lowest validation MSE were used [20]. Early stopping guarded against overfitting. Tuning of the number of hidden units and the window length guarded against underfitting, because a network with too little capacity or too short a memory has a high validation loss and is not selected.

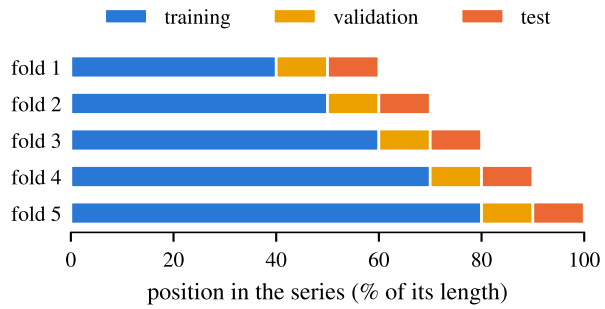


Fig. 3. Training, validation and test blocks of the five cross-validation folds.

B. Cross-Validation

Five expanding-window folds were used, as illustrated in Fig. 3. In each fold, a validation block and a test block of 10% of the series each follow the training block directly. The training block of the first fold contains 40% of the series, and the end of the training block moves forward in equal steps, so that the test block of the fifth fold ends at the last observation. The validation block served both for early stopping and for hyperparameter tuning. The test block was used only once per trained network, after training and tuning were complete.

The input window of a validation or test target reaches back into the previous block. The input values are past observations,

[1]PyTorch: https://pytorch.org
[2]Optuna: https://optuna.org
[3]Code: https://github.com/mattbennett04/DataScience

TABLE III
HYPERPARAMETER SEARCH SPACE

Hyperparameter	Range	Scale
Hidden units q	$[2, 40]$, integer	logarithmic
Window length ℓ	half, one or two times default	categorical
Learning rate η	$[10^{-4}, 3 \times 10^{-2}]$	logarithmic
Weight decay λ	$[10^{-8}, 10^{-3}]$	logarithmic
Batch size	16, 32 or 64	categorical

so the overlap does not leak future information. One-step-ahead forecasts in the validation and test blocks used the true previous observations, which is the standard setting for one-step-ahead evaluation.

C. Hyperparameter Tuning

A separate Optuna study was run for every combination of dataset, network and fold, which gave 75 studies. Each study ran 30 trials. Table III lists the search space. The number of hidden units, the learning rate and the weight decay were sampled on a logarithmic scale, because the effect of these hyperparameters depends on the order of magnitude rather than on the absolute value. The window length was chosen from half, one and two times the default window length of the dataset.

The TPE sampler used the defaults of Optuna 4.9. The first 10 trials of every study were drawn at random. From the 11th trial, the good group contained the best 10% of the trials, rounded up, and TPE evaluated 24 candidates from l to choose each new configuration. The sampler of each fold used a fixed random seed, so that every study is reproducible. The median pruner started to prune only after five trials had completed, and did not prune during the first 10 epochs of the trial. The tuning objective was the lowest validation MSE that early stopping reached. The configuration of the trial with the lowest validation MSE was selected.

A single training run of a network depends on the random initialisation of the weights and the order of the mini-batches. The selected configuration of every fold was therefore retrained from scratch with 10 different random seeds. Each of the 10 retrained networks was evaluated on the test block of the fold. Every dataset and network combination therefore produced $5 \times 10 = 50$ test results.

D. Performance Measures

Let T denote the set of time steps in a test block. The root mean squared error (RMSE), the mean absolute error (MAE) and the symmetric mean absolute percentage error (sMAPE) were computed in the original units of the series,

$$\mathrm{RMSE} = \sqrt{\frac{1}{|T|} \sum_{t \in T} (y_t - \hat{y}_t)^2}, \tag{12}$$

$$\mathrm{MAE} = \frac{1}{|T|} \sum_{t \in T} |y_t - \hat{y}_t|, \tag{13}$$

$$\mathrm{sMAPE} = \frac{100}{|T|} \sum_{t \in T} \frac{2\,|y_t - \hat{y}_t|}{|y_t| + |\hat{y}_t|}. \tag{14}$$

The RMSE served as the primary measure, because the networks were trained on a squared-error loss. The RMSE of a series depends on the scale of the series and on the fold. Hyndman and Koehler recommended relative measures, which divide the error of a model by the error of a benchmark on the same data [12]. The relative RMSE divides the RMSE of a network by the RMSE of the naive forecast on the same test block. A relative RMSE below one means that the network was more accurate than the naive forecast.

E. Determination of the Best Network

The three networks were compared separately for each dataset, following the procedure of Section II-E. A block consisted of one fold and one seed index, which gave $\nu = 50$ blocks per dataset. All three networks were evaluated on the same test block within a block. The Friedman test was applied to the test RMSE at a significance level of 0.05. When the Friedman test rejected the null hypothesis, the three pairwise Wilcoxon signed-rank tests were applied, with the Holm correction. A network was declared the best network on a dataset only when the network had the best mean rank and was significantly better than both other networks. Otherwise, the result was reported as no significant difference between the best networks.

Each network was also compared with the naive forecast. For each dataset and network, the Wilcoxon signed-rank test compared the test RMSE of the network with the RMSE of the naive forecast on the same test block, over the 50 blocks. The Holm correction was applied over all 15 combinations of dataset and network, at a significance level of 0.05.

Across datasets, the mean RMSE of each network per dataset was ranked, and the Friedman test was applied with the five datasets as blocks. With only five blocks, this test has little power to detect a difference.

F. Response to Regime Changes

The response to a regime change was measured on the forecasts from the end of the training block to the end of the series. Only folds in which the break point lay after the training block were used, so that the network had not seen the regime change during training. The absolute percentage error, $\varepsilon_t = |y_t - \hat{y}_t|/|y_t|$, was used instead of the absolute error, because the level of the series changes at the break point. Let t_b denote the break point. The normal error, $\bar{\varepsilon}$, is the mean of ε_t over the 50 steps before t_b. The rolling error, $\tilde{\varepsilon}_t$, is the mean of ε_t over the last five steps up to t, counting only steps from t_b onwards. With the threshold multiplier $\kappa = 2$, two measures were defined. The degradation time, δ_d, is the number of steps from t_b until $\tilde{\varepsilon}_t > \kappa\bar{\varepsilon}$ for the first time. The recovery time, δ_r, is the number of steps from t_b until $\tilde{\varepsilon}_t \leq \kappa\bar{\varepsilon}$ again, after the degradation. A measure that did not occur before the next break point or the end of the series was recorded as censored. The same measures were computed for the naive forecast. The Kruskal-Wallis test compared the recovery times of the three networks.

Algorithm 2 Retraining after a detected change

Set the reference error to the mean absolute percentage error of the network on the validation block
for each time step after the validation block **do**
  Forecast the next value with the current network, then observe the true value
  if no change has been detected and the rolling error over five steps exceeds twice the reference error **then**
    Record the current time step as the detection time
  end if
  if at least 36 steps have passed since the detection, and 12 steps have passed since the last retraining **then**
    Refit the scaling constants on the observations since the detection
    Retrain the network on the observations since the detection, with early stopping on the most recent 20%
  end if
end for

G. Retraining after Detected Drift

The retraining experiment simulated a network in use. For each of D686, M6864 and M26487, the network was deployed after the validation block of the latest fold in which both the training block and the validation block ended before the break point. For M26487, the break point at $t = 606$ was used. The network was trained exactly as in the main experiment, with the tuned configuration of that fold. From the end of the validation block, the network forecast one step at a time, and each true value became known after the forecast was made.

The detector used the absolute percentage error. The reference error was the mean absolute percentage error of the network on the validation block, which is known at the time of deployment. The break point was therefore not used by the detector. The detector fired the first time that the rolling error over the last five steps exceeded $\kappa = 2$ times the reference error. Algorithm 2 describes the retraining that followed. Retraining started once 36 new observations had arrived after the detection, which is three cycles of the monthly series. Retraining was repeated every 12 steps, each time on all observations since the detection. The last 20% of the new windows served as the validation set for early stopping. The scaling constants were refitted at every retraining, with robust scaling and clipping for all three datasets, because the new observations include the jump at the break point.

Three strategies were compared. The first strategy never retrains, which reproduces the main experiment. The second strategy retrains a network with new random weights at every retraining. The third strategy fine-tunes the current weights on the new observations. Two references were added. The naive forecast needs no retraining. A linear autoregressive model on the logarithmic differences, with 12 lags, was refitted by least squares on the same schedule. The linear model was told the true break point, which makes the linear model an optimistic reference. Each strategy was run with 10 seeds for each network. The retrained networks reuse the configuration that was tuned before the break point. The experiment on M6864 was therefore repeated with one shared configuration for all three networks, namely 16 hidden units, a window of 24 steps, a learning rate of 0.005, a weight decay of 10^{-6} and a batch size of 16. The shared configuration separates the effect of the architecture from the effect of the tuned configuration. The RMSE from the break point to the end of the series was compared between strategies and networks with Wilcoxon signed-rank tests over the 10 paired seeds, with the Holm correction for the three pairs of networks.

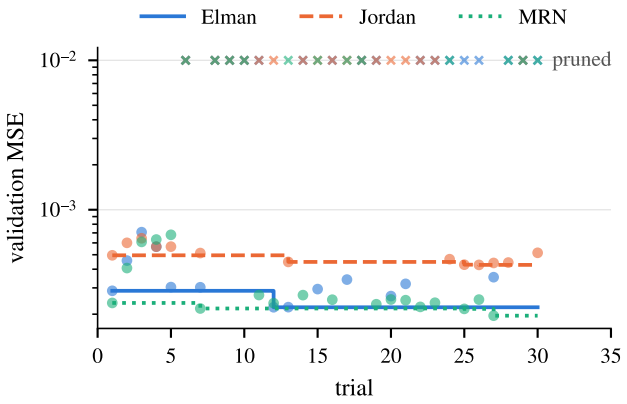


Fig. 4. Validation mean squared error of every trial for H279, fifth fold. Lines show the best value so far, and crosses mark pruned trials.

V. Results and Discussion

This section presents and discusses the results. Section V-A examines the tuning and the control of overfitting. Section V-B reports the forecast accuracy. Section V-C reports the statistical comparison. Section V-D reports the response to regime changes. Section V-E reports the retraining experiment. Section V-F reports the effect of the scaling of M26487. Section V-G discusses the results. The main procedure took about 18 hours of processor time, and the retraining experiment took about five hours.

A. Tuning and Overfitting Control

Fig. 4 shows the 30 trials of the H279 studies of the fifth fold. The first 10 random trials covered a wide range of validation MSE values. The best validation MSE improved during the TPE phase for all three networks, and the pruner stopped the trials that fell behind the median. Over all 75 studies, the median pruner stopped 744 of the 2,250 trials, which is 33%.

Table IV lists the selected hyperparameters. The Elman network and the multi-recurrent network selected more hidden units on H279 than on the other datasets. The Jordan network selected fewer hidden units than the Elman network on four of the five datasets. The multi-recurrent network had by far the most trained weights, with a median of 6,043 weights on H279 compared with 1,331 for the Elman network and 61 for the Jordan network. The selected weight decay was small, with

TABLE IV
SELECTED HYPERPARAMETERS: MEDIAN HIDDEN UNITS, WINDOW LENGTHS SELECTED, MEDIAN NUMBER OF WEIGHTS AND MEDIAN BEST EPOCH

Series	Network	q	ℓ selected	Weights	Epoch
H279	Elman	35	12, 24, 48	1,331	60
	Jordan	15	12, 24, 48	61	52
	Multi-rec.	38	24, 48	6,043	57
W8	Elman	20	52, 104	461	30
	Jordan	9	26, 104	37	18
	Multi-rec.	12	26, 52, 104	661	36
D686	Elman	16	7, 14	305	13
	Jordan	13	7, 14	53	19
	Multi-rec.	21	7, 14	1,912	16
M6864	Elman	7	6, 12	71	33
	Jordan	8	6, 12, 24	33	36
	Multi-rec.	9	6, 12, 24	388	21
M26487	Elman	24	6, 12, 24	649	15
	Jordan	9	6, 12	37	4
	Multi-rec.	10	6, 12, 24	471	17

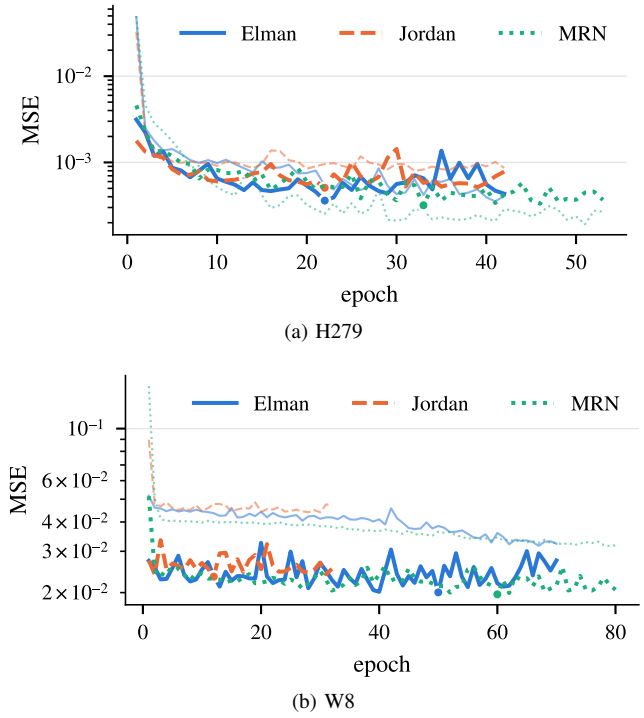
medians over all studies of 1.7×10^{-7} for the Elman network, 3.8×10^{-6} for the Jordan network and 9.3×10^{-6} for the multi-recurrent network. The penalty therefore had little influence, and early stopping was the main control of overfitting.

Fig. 5 shows the training and validation loss of the first retrained network of the fifth fold, for H279 and W8. The training loss of every network continued to decrease after the validation loss had levelled off. Early stopping restored the weights at the minimum of the validation loss, before the gap between the two curves widened. The median training MSE and the median validation MSE at the restored epoch were of the same order of magnitude for every dataset and network. On H279, for example, the medians were 3.0×10^{-4} and 2.9×10^{-4} for the Elman network. The restored networks therefore showed no sign of overfitting. The median restored epoch over all runs was 27, and only five of the 750 retrained networks reached the limit of 300 epochs. The limit therefore did not cut training short.

B. Forecast Accuracy

Table V lists the test errors of the three networks and the naive forecast, and Fig. 6 shows the distribution of the relative RMSE. The standard deviations of the RMSE of D686, M6864 and M26487 are larger than the means. The large standard deviations arise because the test block of one fold contains the break point, and the error at the break point dominates the RMSE of that fold. The relative RMSE removes the differences between folds.

On H279, all three networks were far more accurate than the naive forecast. The mean relative RMSE was 0.14 for the Elman network, 0.18 for the Jordan network and 0.15 for the multi-recurrent network. Fig. 7(a) shows that the forecasts of all three networks followed the daily cycle closely. On W8, the networks improved on the naive forecast by 3% to 6% on average. The boxes of W8 in Fig. 6 extend above one, so the networks were less accurate than the naive forecast on part of



(a) H279

(b) W8

Fig. 5. Training loss, thin lines, and validation loss, thick lines, of the first retrained network of the fifth fold. Dots mark the restored epoch.

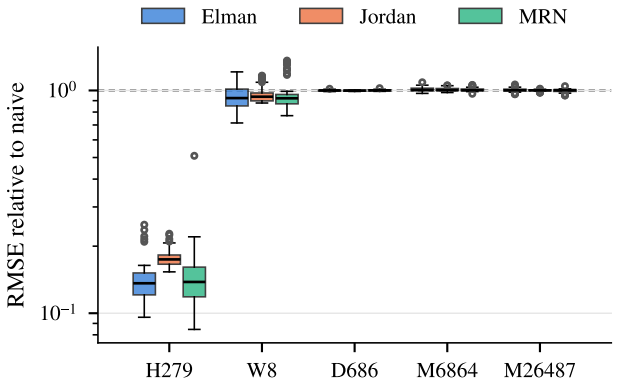


Fig. 6. Test root mean squared error relative to the naive forecast, over five folds and 10 seeds. The dashed line marks equal accuracy.

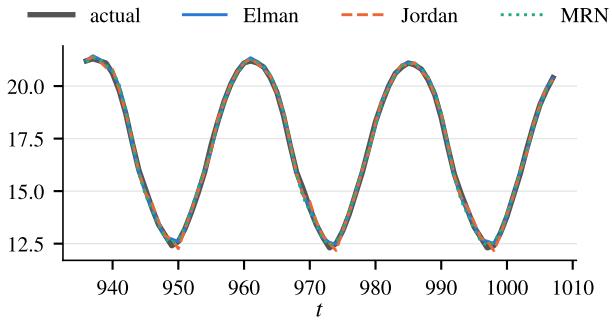
the folds. Fig. 7(b) shows that the forecasts of W8 followed the seasonal level but did not anticipate the single-week spikes. On D686, M6864 and M26487, the mean relative RMSE of every network lay between 0.998 and 1.013. The networks were therefore no more accurate than the naive forecast on the three random-walk-like series.

C. Statistical Comparison

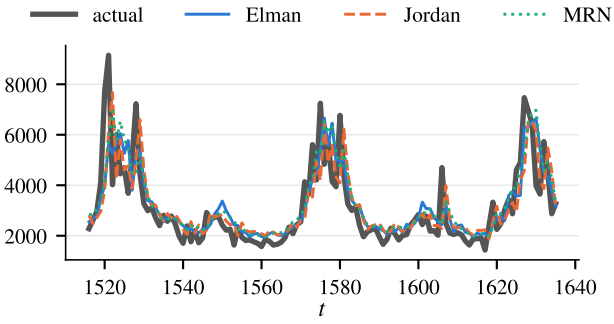
Table VI lists the mean ranks and the results of the tests. On H279, the Friedman test rejected equal performance with $p < 0.001$. The Elman network and the multi-recurrent network were both significantly better than the Jordan network. The difference between the Elman network and the multi-recurrent network was not significant ($p = 0.496$). On W8,

TABLE V
TEST ERRORS OVER FIVE FOLDS AND 10 SEEDS: MEAN ± STANDARD DEVIATION, AND MEAN RELATIVE ROOT MEAN SQUARED ERROR

Series	Network	RMSE	Relative RMSE	MAE	sMAPE (%)
H279	Elman	$\mathbf{0.120 \pm 0.028}$	0.14	0.096 ± 0.022	0.57 ± 0.14
	Jordan	0.150 ± 0.016	0.18	0.118 ± 0.010	0.72 ± 0.06
	Multi-recurrent	0.123 ± 0.053	0.15	0.101 ± 0.047	0.60 ± 0.28
	Naive	0.845	1		
W8	Elman	$\mathbf{1{,}125 \pm 208}$	0.94	714 ± 115	16.4 ± 1.7
	Jordan	$1{,}147 \pm 157$	0.96	718 ± 81	16.5 ± 2.2
	Multi-recurrent	$1{,}160 \pm 250$	0.97	769 ± 175	17.8 ± 2.6
	Naive	1,192	1		
D686	Elman	222.3 ± 256.7	1.00	84.2 ± 35.3	0.93 ± 0.32
	Jordan	$\mathbf{222.1 \pm 256.7}$	1.00	84.0 ± 35.3	0.93 ± 0.32
	Multi-recurrent	222.5 ± 256.6	1.00	84.4 ± 35.3	0.94 ± 0.33
	Naive	222.2	1		
M6864	Elman	$\mathbf{202.6 \pm 383.6}$	1.01	152.9 ± 303.2	3.03 ± 5.45
	Jordan	203.2 ± 384.9	1.01	154.1 ± 305.7	3.05 ± 5.51
	Multi-recurrent	203.0 ± 384.4	1.01	155.0 ± 307.4	3.08 ± 5.56
	Naive	203.0	1		
M26487	Elman	295.1 ± 551.2	1.01	53.2 ± 74.9	0.96 ± 0.84
	Jordan	$\mathbf{294.4 \pm 549.8}$	1.00	53.4 ± 75.1	0.95 ± 0.85
	Multi-recurrent	295.2 ± 551.9	1.00	52.9 ± 74.8	0.95 ± 0.84
	Naive	294.2	1		



(a) H279, last 72 hours



(b) W8, last 120 weeks

Fig. 7. Test forecasts of the fifth fold, averaged over the 10 seeds.

the Friedman test rejected equal performance ($p = 0.027$), but none of the Holm-corrected pairwise tests was significant. The Elman network had the best mean rank on W8.

On D686, the Friedman test and all three pairwise tests were significant, and the Jordan network had the best mean rank. The differences between the mean RMSE values of D686

TABLE VI
MEAN RANKS, FRIEDMAN TEST AND HOLM-CORRECTED PAIRWISE WILCOXON TESTS ON THE TEST ROOT MEAN SQUARED ERROR

	Mean rank			Friedman	Holm-corrected p		
Series	E	J	M	p	E–J	E–M	J–M
H279	1.74	2.70	**1.56**	<0.001	<0.001	0.496	<0.001
W8	**1.78**	2.30	1.92	0.027	0.142	0.142	0.546
D686	2.00	**1.58**	2.42	<0.001	0.006	0.022	<0.001
M6864	2.20	2.08	**1.72**	0.044	0.389	0.389	0.309
M26487	2.16	**1.86**	1.98	0.320	–	–	–
All five	**1.40**	2.00	2.60	0.165			

E: Elman, J: Jordan, M: multi-recurrent.

were, however, smaller than 0.2%, and none of the networks differed from the naive forecast. The significance on D686 reflects the consistency of tiny differences over the 50 blocks and does not indicate a difference of practical importance. On M6864, the Friedman test was only just significant ($p = 0.044$), and no pairwise test was significant. On M26487, the Friedman test did not reject equal performance.

Across the five datasets, the Elman network had the best mean rank of 1.40, followed by the Jordan network with 2.00 and the multi-recurrent network with 2.60. The Friedman test over the five datasets did not reject equal performance ($p = 0.165$).

Table VII lists the comparison of each network with the naive forecast. On H279, all three networks were more accurate than the naive forecast in all 50 blocks (Holm-corrected $p < 0.001$). On W8, the Elman network ($p = 0.017$) and the Jordan network ($p = 0.029$) were significantly more accurate than the naive forecast. The multi-recurrent network was more accurate in 40 of the 50 blocks, but the difference was not

TABLE VII
COMPARISON OF EACH NETWORK WITH THE NAIVE FORECAST OVER 50 BLOCKS

Series	Network	Relative RMSE	More accurate	Holm p
H279	Elman	0.142	50 of 50	<0.001
	Jordan	0.178	50 of 50	<0.001
	Multi-rec.	0.145	50 of 50	<0.001
W8	Elman	0.941	37 of 50	0.017
	Jordan	0.962	40 of 50	0.029
	Multi-rec.	0.971	40 of 50	0.26
D686	Elman	0.999	19 of 50	0.53
	Jordan	0.998	30 of 50	0.53
	Multi-rec.	1.002	18 of 50	0.029
M6864	Elman	1.013	15 of 50	0.052
	Jordan	1.010	13 of 50	0.072
	Multi-rec.	1.009	12 of 50	0.002
M26487	Elman	1.005	17 of 50	0.052
	Jordan	1.001	13 of 50	0.008
	Multi-rec.	0.998	22 of 50	0.53

TABLE VIII
MEDIAN DEGRADATION AND RECOVERY TIMES IN STEPS, WITH THE NUMBER OF CENSORED RECOVERY TIMES

Break point	Forecaster	δ_d	δ_r	Censored
D686, $t = 3{,}626$	Elman	0	17	0 of 50
	Jordan	0	17	0 of 50
	Multi-recurrent	0	17	0 of 50
	Naive	0	17	0 of 5
M6864, $t = 2{,}430$	Elman	0	> 247	50 of 50
	Jordan	0	> 247	50 of 50
	Multi-recurrent	0	> 247	50 of 50
	Naive	0	> 247	5 of 5
M26487, $t = 606$	Elman	0	5	0 of 30
	Jordan	0	5	0 of 30
	Multi-recurrent	0	5	0 of 30
	Naive	0	5	0 of 3

significant after the Holm correction ($p = 0.26$). On the three random-walk-like series, no network was significantly more accurate than the naive forecast. The multi-recurrent network on D686 and M6864 and the Jordan network on M26487 were significantly less accurate, by 0.1% to 0.9%. The networks therefore improved on the naive forecast only on the series with a repeating pattern.

D. Response to Regime Changes

Table VIII lists the degradation and recovery times, and Fig. 8 shows the mean rolling error ratio after each break point. The break point at $t = 303$ of M26487 lay inside the training block of every fold and was therefore excluded. The degradation time was zero steps for every network and for the naive forecast at every break point. All three regime changes are single-step jumps. The first forecast after a jump misses the full size of the jump, because no one-step-ahead forecast based on past values anticipates the jump. The degradation time therefore does not distinguish between the networks on these datasets.

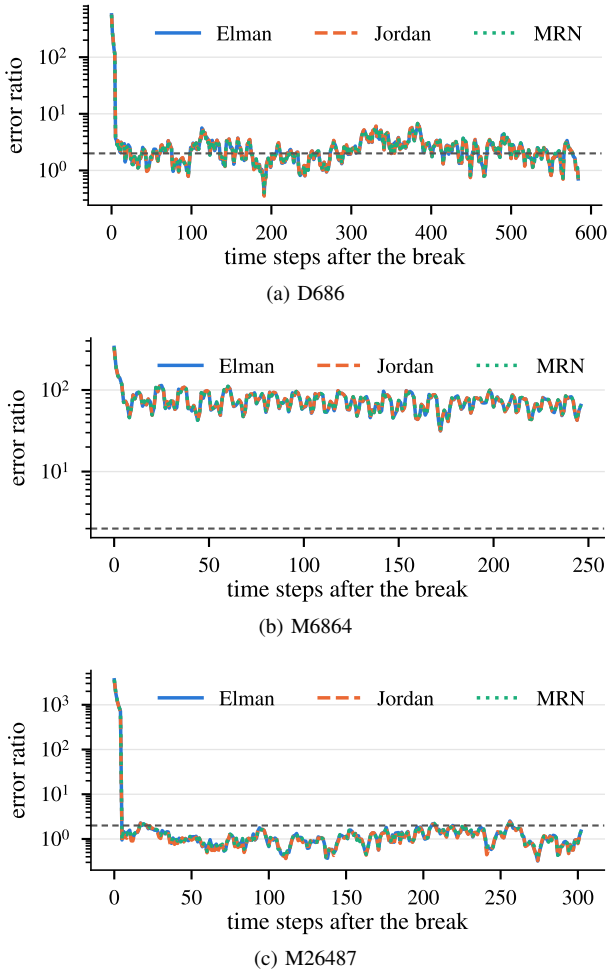


Fig. 8. Rolling absolute percentage error after the break point, divided by the error before the break point. The dashed line marks the threshold $\kappa = 2$.

The recovery time did distinguish between the break points. After the fall of D686, the error of every network returned below the threshold after a median of 17 steps. After the fall of M26487, the error returned after five steps. The naive forecast recovered after the same number of steps in both cases. After differencing, the networks forecast relative changes, so they adapted to the new level as soon as it entered the input window. The Kruskal-Wallis test found a significant difference between the recovery times on D686 ($p < 0.001$), but the medians of the three networks were identical. After the regime change of M6864, the error of every network and of the naive forecast stayed between about 30 and 100 times the error before the break, for all 247 remaining steps. The new regime of M6864 contains large oscillations that did not occur in any training block. None of the networks recovered, because none of them had seen the new dynamics during training.

E. Retraining after Detected Drift

On M6864, the detector fired at the break point in the median run, and in at most one of the 10 runs per network before the break point. Table IX lists the RMSE after the break point relative to the naive forecast. Without retraining, every

TABLE IX
ROOT MEAN SQUARED ERROR AFTER THE BREAK POINT OF M6864, RELATIVE TO THE NAIVE FORECAST: MEAN ± STANDARD DEVIATION OVER 10 SEEDS

		Retraining		
Settings	Network	None	New weights	Fine-tuned
Tuned	Elman	1.00 ± 0.02	$\mathbf{0.51 \pm 0.01}$	0.60 ± 0.16
	Jordan	1.00 ± 0.02	0.97 ± 0.01	0.98 ± 0.02
	Multi-rec.	1.00 ± 0.02	0.63 ± 0.03	0.68 ± 0.05
Shared	Elman	1.00 ± 0.01	0.53 ± 0.04	0.58 ± 0.03
	Jordan	1.00 ± 0.01	0.98 ± 0.01	0.98 ± 0.02
	Multi-rec.	1.00 ± 0.01	$\mathbf{0.51 \pm 0.02}$	0.53 ± 0.02
Linear, told the break point		0.43		

network matched the naive forecast, as in the main experiment. Retraining with new weights reduced the error of the Elman network to 0.51 of the naive error and the error of the multi-recurrent network to 0.63. The Jordan network improved only to 0.97. Retraining was significantly better than no retraining for all three networks ($p \leq 0.006$).

The tuned configurations explain part of the differences. Before the break point, M6864 behaved like a random walk, and the tuning selected small configurations, because no configuration improved on the naive forecast. The tuned Jordan network had two hidden units, and the tuned multi-recurrent network had a window of six steps, which is shorter than the 12-step cycle of the new regime. With the shared configuration, the multi-recurrent network improved to 0.51 and did not differ significantly from the Elman network with 0.53 ($p = 0.232$). The Jordan network still reached only 0.98 and was significantly worse than both other networks ($p = 0.006$). The failure of the Jordan network is therefore a property of the architecture and not of the tuned configuration.

Fig. 9 shows the rolling error of the shared configuration after the break point. All forecasts had a rolling error of about 17% until the first retraining, 36 steps after the break point. After the first retraining, the error of the Elman and multi-recurrent networks fell to about 5%, close to the linear model, while the error of the Jordan network stayed with the naive forecast. New weights gave a lower mean error than fine-tuning for the Elman and multi-recurrent networks. The difference was significant for the multi-recurrent network ($p = 0.037$) but not for the Elman network ($p = 0.065$) with the shared configuration. Fine-tuning starts from weights that were fitted to the previous scaling constants, which changed at every retraining.

On D686 and M26487, retraining neither helped nor harmed. The relative RMSE of all strategies lay between 1.000 and 1.004. The detector fired in every run before the break point, after 8 steps on D686 and 20 steps on M26487, so every detection on these series was a false alarm. The error on the validation block was lower than the typical error after deployment, so the threshold of twice the validation error was too sensitive for these noisy series. The false alarms did no harm, because the new observations of a random walk contain

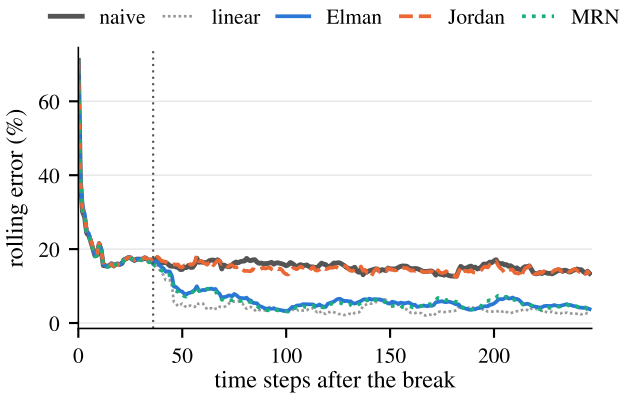


Fig. 9. Rolling absolute percentage error over 12 steps after the break point of M6864, with retraining from new weights and the shared configuration. The dotted line marks the first retraining.

TABLE X
TEST ROOT MEAN SQUARED ERROR AND MEDIAN RECOVERY TIME ON M26487 FOR TWO SCALING CHOICES

	Raw level, min-max		Log, differenced, robust	
Forecaster	RMSE	δ_r	RMSE	δ_r
Elman	367.0	> 303 (23 of 30)	295.1	5
Jordan	385.2	> 303 (19 of 30)	294.4	5
Multi-recurrent	428.1	36 (11 of 30)	295.2	5
Naive	294.2	5	294.2	5

Brackets give the number of censored recovery times.

nothing new to learn.

F. Effect of the Scaling of M26487

The complete procedure was first run with min-max scaling on the raw level of M26487, without the logarithm and without differencing. Table X compares the first run with the final run. With scaling on the raw level, every network was less accurate than the naive forecast, by 25% for the Elman network, 31% for the Jordan network and 46% for the multi-recurrent network. The training blocks of M26487 contain the level of about 13,000, while the last regime lies at about 700. After the fall at $t = 606$, all later values were compressed into less than 1% of the scaled range, and the networks could not resolve changes of that size. Most networks with raw-level scaling also did not recover from the fall at $t = 606$ within the 303 remaining steps. With the logarithm, differencing and robust scaling, all three networks matched the naive forecast and recovered after five steps.

G. Discussion

The results separate the datasets into two groups. H279 and W8 contain repeating patterns that the values in the input window predict. The networks learned these patterns and improved on the naive forecast. On D686, M6864 and M26487, a relative RMSE of about one indicates that the input window contained almost no information about the next change of the series. The best one-step forecast of a random walk is the last value [13], and the tuned networks

converged to that forecast. The early restored epochs of these datasets support this interpretation, with a median restored epoch of four for the Jordan network on M26487, as listed in Table IV. A network that outputs a scaled change of about zero reproduces the naive forecast, and further training on noise only increased the validation loss. The comparison between the networks is therefore only informative on H279 and W8.

On H279, the Jordan network was significantly less accurate than the other two networks. The Jordan context is a single number per step, while the Elman context holds q numbers and the multi-recurrent context holds $pq + p$ numbers. The input enters the network one value per step. In the Jordan network, the only path from the earlier steps of the window to the current step is the single context unit. The result is consistent with this single unit limiting how much information about the earlier part of the window reaches the last step. The Jordan network also selected fewer hidden units, which kept the number of trained weights low. On W8, the high noise level hid differences of this size, and no pair of networks differed significantly.

The multi-recurrent network did not outperform the Elman network on any dataset. On H279, the multi-recurrent network had about 4.5 times as many trained weights as the Elman network. Its additional memory banks provide information from further back within the window. Every window was, however, processed from an empty context, and the window lengths selected on H279 and W8 covered at least half a seasonal period. The results indicate that the slow memory banks added little information beyond the information in the input window. The multi-recurrent network also had the largest spread of relative RMSE on H279 and W8, as Fig. 6 shows, which indicates a greater sensitivity to the random initialisation.

The response to a level shift was determined by the pre-processing rather than by the architecture. With differencing, all networks recovered from a level shift as fast as the naive forecast. With raw-level scaling, the networks failed to recover. A change in dynamics was different. Without retraining, no network recovered from the change of M6864, because none of the training blocks contained examples of the new dynamics. With informed adaptation in the sense of Gama *et al.* [10], the architecture mattered again. The Elman and multi-recurrent networks learned the new 12-step cycle from 36 to 247 new observations, while the Jordan network did not. The Jordan network was also the weakest network on H279, the other series with a regular cycle. Both results are consistent with the single context unit of the Jordan network limiting the information that reaches the last step of the window. The retraining experiment also showed that a configuration tuned on the old regime limits what retraining achieves, because the tuned Jordan and multi-recurrent networks were smaller than the new regime required.

The study has five main limitations. First, each network was tuned with 30 trials per fold, so better configurations could exist. Second, the self-connection weight of the Jordan network and the bank ratios of the multi-recurrent network were fixed and not tuned. Third, the context layer was reset for every window, so no network carried memory beyond the window. Fourth, the conclusions rest on five series, which gives the Friedman test over datasets little power. Fifth, the retraining experiment used a single detector threshold, which produced false alarms on the two random-walk-like series, and only M6864 had a new regime with a learnable pattern.

VI. Conclusion

This report compared three simple recurrent neural networks, the Elman, Jordan and multi-recurrent networks, for one-step-ahead forecasting of five M4 series, with expanding-window cross-validation and hyperparameters tuned by Optuna on the validation block of every fold.

On the hourly series H279, the Elman network, with a root mean squared error (RMSE) of 0.120, and the multi-recurrent network, with an RMSE of 0.123, were significantly more accurate than the Jordan network, with an RMSE of 0.150. The difference between the Elman network and the multi-recurrent network was not significant. All three networks reduced the error of the naive forecast by more than 80% on H279 and by 3% to 6% on the weekly series W8. On the three random-walk-like series, the networks matched the naive forecast but did not improve on it. Across all five datasets, the Elman network had the best mean rank, but no network was significantly better than the others. The Elman network was therefore the most consistent choice, with fewer trained weights than the multi-recurrent network on four of the five datasets.

After abrupt level shifts, all networks recovered as fast as the naive forecast when the series was log-transformed and differenced. With scaling on the raw level of M26487, all networks were 25% to 46% less accurate than the naive forecast and mostly did not recover. No network recovered from the change in dynamics of M6864 without retraining. Retraining triggered when the rolling error exceeded twice the validation error halved the RMSE of the Elman and multi-recurrent networks after that change, while the Jordan network improved by only 2% to 3%. The response to a level shift was therefore determined by the pre-processing, while the ability to adapt to new dynamics depended on retraining and on the architecture. Future work could carry the context across windows, tune the memory parameters of the Jordan and multi-recurrent networks, re-tune the hyperparameters after a detected change, and calibrate the detector threshold to reduce false alarms.

References

[1] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631.

[2] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192–213, 2012.

[3] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Proceedings of the 25th Conference on Neural Information Processing Systems*, 2011, pp. 2546–2554.

[4] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.
[5] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.
[6] D. A. Dickey and W. A. Fuller, "Distribution of the estimators for autoregressive time series with a unit root," *Journal of the American Statistical Association*, vol. 74, no. 366a, pp. 427–431, 1979.
[7] J. L. Elman, "Finding structure in time," *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990.
[8] A. P. Engelbrecht, *Computational Intelligence: An Introduction*, 2nd ed. Chichester, United Kingdom: John Wiley & Sons, 2007.
[9] M. Friedman, "The use of ranks to avoid the assumption of normality implicit in the analysis of variance," *Journal of the American Statistical Association*, vol. 32, no. 200, pp. 675–701, 1937.
[10] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.
[11] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
[12] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, 2006.
[13] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd ed. Melbourne, Australia: OTexts, 2018.
[14] M. I. Jordan, "Attractor dynamics and parallelism in a connectionist sequential machine," in *Proceedings of the Eighth Annual Conference of the Cognitive Science Society*, 1986, pp. 531–546.
[15] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proceedings of the 3rd International Conference on Learning Representations*, 2015.
[16] D. Kwiatkowski, P. C. B. Phillips, P. Schmidt, and Y. Shin, "Testing the null hypothesis of stationarity against the alternative of a unit root," *Journal of Econometrics*, vol. 54, no. 1–3, pp. 159–178, 1992.
[17] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, "Efficient backprop," in *Neural Networks: Tricks of the Trade*, ser. Lecture Notes in Computer Science, vol. 1524. Berlin, Germany: Springer, 1998, pp. 9–50.
[18] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M4 competition: 100,000 time series and 61 forecasting methods," *International Journal of Forecasting*, vol. 36, no. 1, pp. 54–74, 2020.
[19] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proceedings of the 30th International Conference on Machine Learning*, 2013, pp. 1310–1318.
[20] L. Prechelt, "Early stopping – but when?" in *Neural Networks: Tricks of the Trade*, ser. Lecture Notes in Computer Science, vol. 1524. Berlin, Germany: Springer, 1998, pp. 55–69.
[21] L. J. Tashman, "Out-of-sample tests of forecasting accuracy: An analysis and review," *International Journal of Forecasting*, vol. 16, no. 4, pp. 437–450, 2000.
[22] C. Ulbricht, "Multi-recurrent networks for traffic forecasting," in *Proceedings of the Twelfth National Conference on Artificial Intelligence*, vol. 1, 1994, pp. 883–888.
[23] P. J. Werbos, "Backpropagation through time: What it does and how to do it," *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1550–1560, 1990.
[24] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

Appendix A
Acronyms

ADF	augmented Dickey-Fuller
KPSS	Kwiatkowski-Phillips-Schmidt-Shin
MAE	mean absolute error
MSE	mean squared error
RMSE	root mean squared error
sMAPE	symmetric mean absolute percentage error
TPE	tree-structured Parzen estimator

Appendix B
Symbols

$\mathbf{A}$	weights of the output context banks
$\mathbf{a}$	context weights of the Jordan network
$\mathbf{b}$	hidden biases
$\mathbf{c}_\tau$	context vector of the Elman network
$\mathbf{c}_\tau^{(i)}$	hidden context bank i
χ_F^2	Friedman statistic
d	output bias
δ_d	degradation time
δ_r	recovery time
e_τ	context unit of the Jordan network
$e_\tau^{(i)}$	output context bank i
$\mathbf{e}_\tau$	vector of the output context banks
ε_t	absolute percentage error
$\bar{\varepsilon}$	mean error before the break point
$\tilde{\varepsilon}_t$	rolling error after the break point
η	learning rate
f	validation loss of a configuration
f^*	threshold between good and poor trials
$\boldsymbol{\phi}$	hyperparameter configuration
g	density of poor configurations
γ	fraction of good trials
$\mathbf{h}_\tau$	hidden layer
i	index of a memory bank
j	index of a model
κ	threshold multiplier
l	density of good configurations
ℓ	window length
λ	weight decay coefficient
μ	self-connection weight of the Jordan network
n	length of the series
n_{in}	number of inputs to a layer
ν	number of blocks
o_τ	output
p	number of memory banks
q	number of hidden units
$\bar{r}_j$	mean rank of model j
ρ_i	ratio of memory bank i
s	epoch at which a trial is pruned
T	set of test time steps
t	time step
t_b	break point
τ	step within the input window
$\boldsymbol{\theta}$	vector of all trained weights
$\mathbf{U}$	context weights of the Elman network
$\mathbf{U}_i$	weights of hidden context bank i
$\mathbf{v}$	output weights
$\mathbf{w}$	input weights
x_τ	input at step τ
y_t	observed value
$\hat{y}_t$	forecast
$\tilde{y}_t$	transformed value
$\tilde{y}_{\text{lo}}, \tilde{y}_{\text{hi}}$	scaling constants
z_t	scaled value